

Lab - 1:

Experiment No: 1

AIM: Cryptography in Blockchain, Merkle root Tree Hash

Lab Objectives: To explore Blockchain concepts.

Lab Outcomes (LO): Creating Cryptographic Hash using Merkle Tree (LO1)

Task to be performed :

1. Make a copy of this [Google Colab Notebook](#)
2. Try to solve the errors in each of the 4 Programs
3. In the 4th Program - Constructing a Merkle Tree Root Hash, modify the code as follows:

- Update the transactions list with valid entries.

eg : transactions = ['A -> B : \$10', 'B -> C : \$5', 'C -> A : \$2']

Sample Transactions to be considered

T1 : Alice → Bob : \$200; T2 : Bob → Dave : \$500; T3 : Dave → Eve : \$100

T4 : Eve → Alice : \$300; T5 : Roo → Bob : \$50

- Hash the transactions before combining them in the for loop
- Print all the intermediate hash during the construction of the Merkle Tree Root Hash

Tools & Libraries used :

- Python Libraries: **hashlib**

Instructions : [\(Prepare for viva for the following topics\)](#)

1. Cryptographic Hash functions in Blockchain
2. What is a Merkle Tree?
3. What is a Cryptographic Puzzle and explain the Golden Nonce
4. How does a Merkle Tree work?
5. Benefits of Merkle Tree
6. Use cases of Merkle Tree

Outcome :

1. Understood the concept of hashing, Cryptographic Puzzle to find nonce, Merkle Tree and its relevance.
2. Implemented programs
 - To find the Golden Nonce while solving the Cryptographic Puzzle
 - To construct a Merkle Tree for the given transactions
3. Prepare a document with Aim, Tasks performed, Program, Output and Conclusion.
4. Submit the hardcopy **by the 1st week of Sem**
(As per the instructions, submit a hard copy of the same).

Aim: Cryptography in Blockchain, Merkle root Tree Hash

Tasks Performed:

1. **Hash Generation using SHA-256:**
 - o Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.
2. **Target Hash Generation with Nonce:**
 - o Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.
3. **Proof-of-Work Puzzle Solving:**
 - o Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.
4. **Merkle Tree Construction:**
 - o Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Program and Output

[Blockchain_Lab_1.ipynb](#)

Program 1: Python program that uses the hashlib library to create the hash of a given string:

XXXXXXX #error statement; 3 Error

```
import hashlib
#error statement

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()
    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))
    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()
#error statement
    # Return the hash string
    return hash_string

# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)
#error statement
```

Enter a string: g

Hash: cd0aa9856147b6c5b4ff2b7dfee5da20aa38253099ef1b4a64aced233c9afe29

[]

Program #2: Program to generate required target hash with input string and nonce

```
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce
# Error

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)
```

Enter a string: S

Enter the nonce: 3

Hash Code:

44d6a8a73eddb284d49799fdbfa2919a004ece6d2df3546eaa4e246048bcd81

Program #3: Python code for Solving Puzzle for leading zeros with expected nonce and given string 3 Error

```
import hashlib
```

```

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)
#Error
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of
        leading zeros
        if hash_code.startswith(hash_prefix):
#Error
            print("Hash:", hash_code )
            return nonce

        nonce += 1

# Get user input
input_string = "B"
num_zeros = 2

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)
#Error

```

Hash: 00ceeab4df7002aad9c625c69e81c47d07f71027ad3600787a73c7e91fdf8d61

Input String: B

Leading Zeros: 2

Expected Nonce: 18

Program 4: Generating Merkle Tree for given set of Transactions

```
import hashlib
def hash_transaction(transaction):
    return
hashlib.sha256(transaction.encode('utf-8')).hexdigest()
def build_merkle_tree(transactions):
    if len(transactions) == 0:
        return None
    # Step 1: Hash each transaction individually
    transactions = [hash_transaction(tx) for tx in transactions]

    # Print individual transaction hashes
    print("\nIndividual Transaction Hashes:")
    for i, tx_hash in enumerate(transactions):
        print(f"Transaction {i + 1}: {tx_hash}")

    # Step 2: Keep combining hashes until only one remains
    while len(transactions) > 1:

        # If odd number of hashes, duplicate the last one
        if len(transactions) % 2 != 0:
            transactions.append(transactions[-1])

        new_transactions = []

        # Combine pairs and hash them
```

```
for i in range(0, len(transactions), 2):
    combined = transactions[i] + transactions[i + 1]
    hash_combined = hashlib.sha256(
        combined.encode('utf-8')
    ).hexdigest()
    new_transactions.append(hash_combined)
transactions = new_transactions
return transactions[0]
```

```
# Transactions
```

```
transactions = [
    "Soham -> Sam: $300",
    "Shiv -> Mike: $200",
    "Mike -> Siv: $300",
    "Som -> Bob: $30000"
]
```

```
# Build Merkle Tree
```

```
merkle_root = build_merkle_tree(transactions)
```

```
print("\nMerkle Root:", merkle_root)
```

Individual Transaction Hashes:

Transaction 1:

2c31c8c6723700fef3d5b7dcd28b00a35d9e182198c4424f4f01b added1892c9df

Transaction 2:

6bf1499b3a58e0b23fa7a4fba1363d8bf48c4386dd28c613cce63920f0510c9c

Transaction 3:

10233f100e77e6cd97495f3b493eee889a6f1becebaee376df26321295f224b0

Transaction 4:

bfe4a9a41b19e3a83276ee4f8f4f7ba2ce0feb115827dc186effd6c694441960

Merkle Root:

974b94b427c46f3b1c6a976fd79f11cb643c0f80845b917cf4f28c8615b9ff3d

Conclusion:

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- **SHA-256 Hashing** provides a secure and unique fingerprint for any input, ensuring data integrity.
- **Nonce and Target Hashing** allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).
- **Proof-of-Work Puzzle Solving** shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
- **Merkle Tree Construction** efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.